

Note Technique – API de Segmentation Sémantique pour Véhicule Autonome

1. Introduction

La segmentation sémantique d'image est une technique d'analyse visuelle qui consiste à attribuer une étiquette sémantique à chaque pixel d'une image. Contrairement à la classification classique, qui assigne une étiquette à l'image entière, ou à la détection d'objets, qui encadre des objets dans des boîtes, la segmentation identifie des zones sans en distinguer les éléments la constituent, cela permet une compréhension très fine de la scène. Cette méthode est particulièrement pertinente pour les systèmes de vision dans le domaine de la conduite autonome, car elle permet d'identifier précisément les routes, véhicules, piétons, obstacles et éléments du décor.

2. Contexte du projet

Ce projet a pour objectif de concevoir, entraîner, optimiser et déployer un modèle de segmentation sémantique capable de s'intégrer dans une chaîne de traitement d'un système embarqué pour véhicule autonome. Le pipeline complet inclut la préparation des données, l'entraînement du modèle, l'évaluation des performances, le déploiement d'une API de prédiction et le développement d'une interface web de visualisation.

3. Jeux de données

Deux jeux de données ont été utilisés : Cityscapes pour l'entraînement et la validation, et CamVid pour les tests. Les images ont été redimensionnées à 256x256 pixels et les masques convertis au format one-hot encoding avec 8 classes métier pertinentes (route, bâtiment, objet, nature, ciel, humain, véhicule, vide).

Nom	Rôle	Nb images	Taille initiale	Masques	Commentaires
Cityscapes	Entraînement / Validation	2975 / 500	2048x1024	Masques fins (8 classes)	Séquences de scènes urbaines dans environ 50 villes (Allemagne)
CamVid	Test	~700	960x720	Masques réduits (11->8 classes)	Jeu externe, vues urbaines captées en Angleterre

Source d'information sur les données : [arXiv:1604.01685v2 \[cs.CV\]](#) 7 Apr 2016 « The cityscapes dataset for semantic urban scene understanding ».

Il est important de noter que la répartition des classes de ce type d'images sera très déséquilibrée.

	CLASSES	train_class_weights
0	void	0.047312
1	flat	0.012541
2	construction	0.022235
3	object	0.275403
4	nature	0.032225
5	sky	0.137174
6	human	0.406959
7	vehicle	0.066151

Poids moyens sur l'ensemble du jeu d'entraînement

4. Architecture et entraînement du modèle

La démarche d'entraînement repose sur une fonction dédiée qui orchestre l'ensemble du processus. Cette fonction appelle un `DataGenerator` personnalisé, capable de fournir à la volée des images et masques prétraités avec leurs poids associés. Ce générateur personnalisé prend en charge les augmentations, les masques de poids dynamiques et les encodages adaptés. La fonction d'entraînement intègre également un ensemble de `callbacks` tels que l'early stopping, la sauvegarde du meilleur modèle (checkpoint), et la réduction automatique du taux d'apprentissage. L'intégralité des logs (perte, métriques, courbes) est enregistrée dans TensorBoard pour un suivi visuel précis. À l'issue de l'entraînement, la fonction évalue le modèle sur un jeu de test indépendant (CamVid) afin de fournir une estimation robuste de la performance sur des données non vues.

5. Évaluation et métriques

Les performances du modèle sont évaluées principalement à l'aide de la métrique Intersection over Union (IoU), qui mesure le recouvrement entre les pixels prédits et les pixels réels pour chaque classe. Le modèle optimise également des fonctions de perte adaptées à la segmentation :

- Entropie croisée (baseline)
- Dice Loss et Dice Loss pondérée
- Tversky Loss (favorise les classes rares)
- Focal Loss (accentue les erreurs sur classes sous-représentées)

Une attention particulière a été portée à la détection des classes rares telles que « human » et « object ». Le meilleur modèle a atteint un IoU moyen de 0.7347, avec une IoU de 0.4418 pour la classe « human ».

6. Optimisations et variantes de modèles

L'optimisation du modèle de base (U-Net + VGG16) a été réalisée par essais successifs sur trois axes :

- Perte : test de plusieurs fonctions (Dice, Focal, etc.)
- Données : augmentations, poids dynamiques, batch réduit
- Backbone : VGG16, ResNet50/101, MobileNetV2, EfficientNetB3

Des modèles alternatifs ont également été explorés pour comparaison :

- FPN : segmentation hiérarchique avec MobileNetV2
- PSPNet : segmentation multi-échelle avec ResNet50
- DeepLabV3+ : backbone Xception, très prometteur

7. Déploiement de l'API (swagger : [FastAPI - Swagger UI](#))

Une API REST a été développée avec FastAPI, permettant de soumettre une image et d'obtenir le masque segmenté en réponse. La route principale `/predict` accepte une image (optionnellement un masque réel-attention sur le swagger bien décocher "send empty value") et retourne :

- Le masque segmenté colorisé
- Le temps de traitement
- L'IoU si un masque GT est fourni
- Un code d'erreur le cas échéant

L'API fonctionne sur un environnement Docker CPU avec TensorFlow 2.17. Elle est accessible via Swagger UI et testée localement avant déploiement sur une instance EC2.

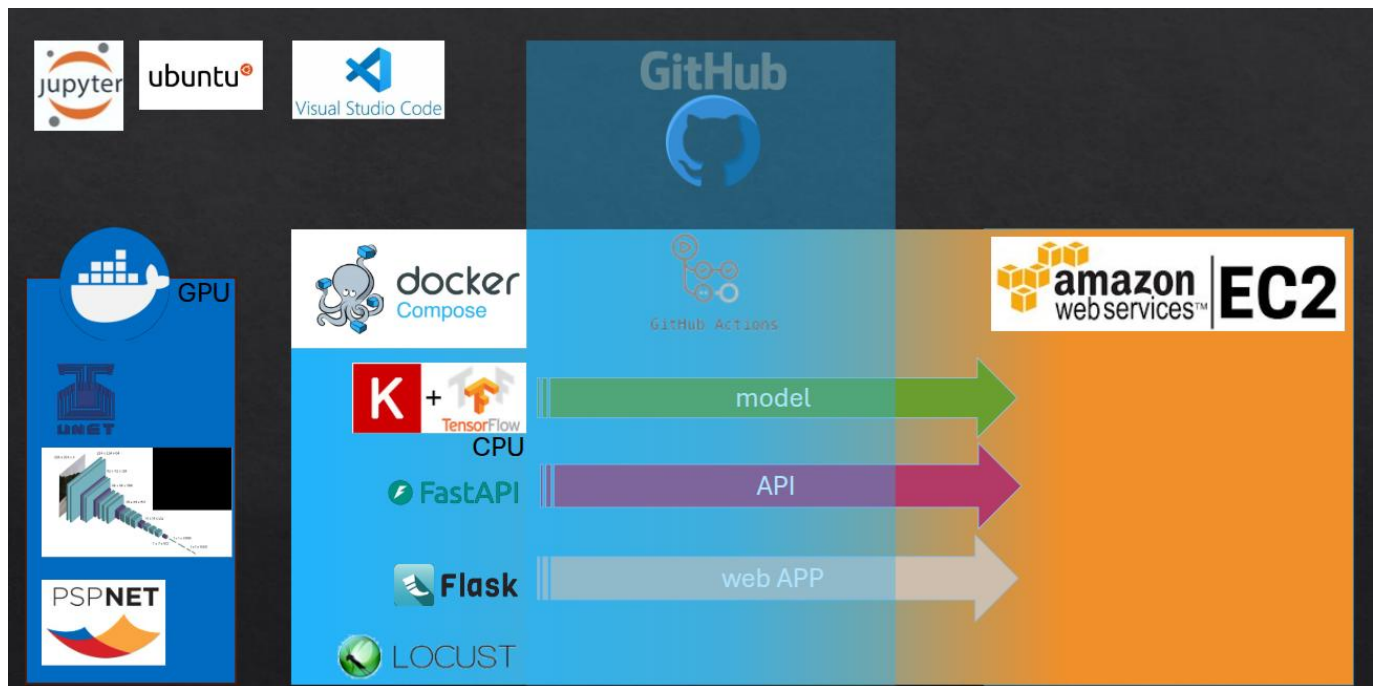
8. Interface Web (WebApp Segmentation d'Images)

Une interface web simple a été développée en Flask pour consommer l'API. Elle permet à l'utilisateur :

- De charger une image
- De visualiser le masque de prédiction avec superposition
- D'obtenir l'IoU et le temps de traitement

L'interface intègre une légende dynamique, une mise en cache des masques réels, un ajustement du ratio de taille du masque fourni. Elle est également déployée sur EC2.

9. Infrastructure et déploiement



Pipeline de déploiement (CI/CD)

Le projet est conteneurisé avec Docker et orchestré via Docker Compose (services `api` et `webapp`). Le code est versionné sur GitHub et le déploiement est automatisé via GitHub Actions : chaque push sur `main` déclenche une mise à jour sur l'instance EC2 distante.

L'instance EC2 utilise Ubuntu, et héberge les deux services sur des ports différents. On a aussi la possibilité d'exécuter locust pour les tests de charge sur un port dédié. Le modèle est stocké localement sur la machine distante pour une latence minimale.

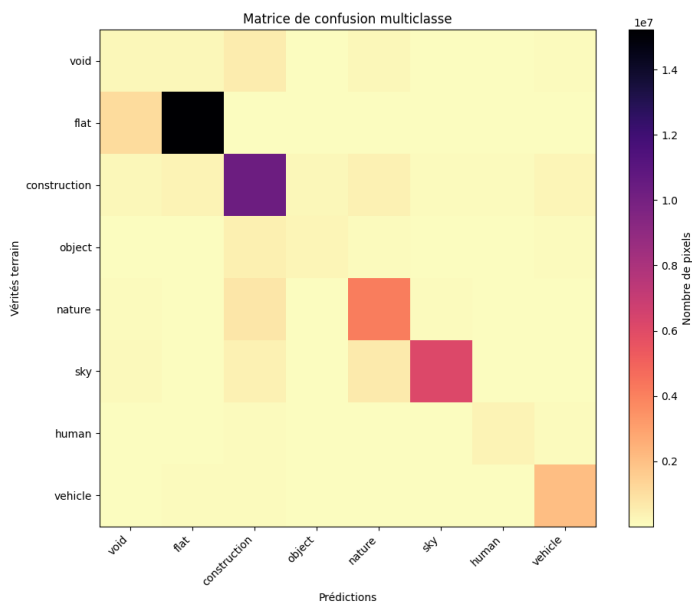
10. Perspectives et améliorations

Des pistes d'amélioration sont envisagées pour une mise en production embarquée :

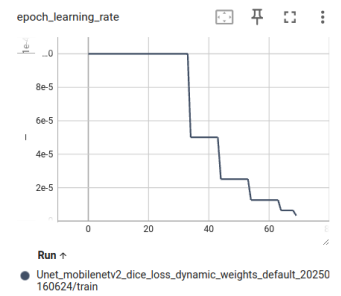
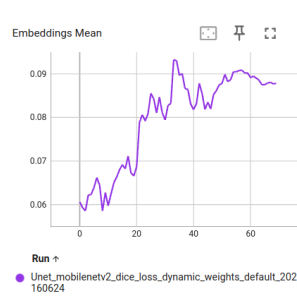
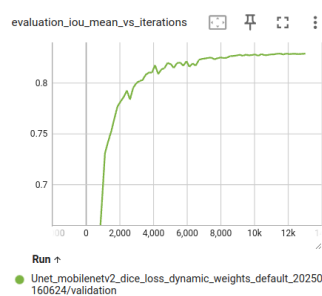
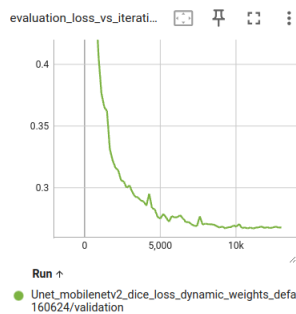
- Conversion du modèle vers ONNX ou TensorRT (instance avec GPU)
- Support du streaming vidéo ou flux caméra en temps réel (une route de traitement par batch est déjà prévue dans l'API)
- Optimisation pour dispositifs embarqués (Jetson / Edge TPU)
- Optimisation du générateur de données (création d'images représentant les classes rares, utilisation d'outils avancés de compensation du déséquilibre de classes), exploration d'un modèle élaboré (recommandation : DeepLabV3+)

Annexe 1 : Tableau récapitulatif des essais et modèle retenu

Modèle	Backbone	Perte	Variante Train	Training time (sec)	Meilleure Epoch	IoU Validation	Loss Validation	IoU Test	IoU Human	IoU Object	Temps Inférence Moyen (ms)
Unet	vgg16	dice_loss_weighted	default	8460	74	0.8444	0.5148	0.6859	0.3764	0.2447	21
Unet	vgg16	dice_loss	smaller_batch	15513	58	0.8434	0.2402	0.7001	0.4414	0.2512	24
Unet	vgg16	dice_loss	dynamic_weights	14206	54	0.8423	0.2434	0.6802	0.4315	0.2402	22
Unet	vgg16	dice_loss	no_augmentation	9340	34	0.8421	0.2476	0.7096	0.4174	0.2292	21
Unet	vgg16	dice_loss	no_weights	14354	68	0.8415	0.2452	0.6633	0.3387	0.2459	20
Unet	vgg16	dice_loss	default	12600	56	0.8393	0.2487	0.6721	0.4040	0.2419	20
Unet	efficientnetb3	dice_loss	dynamic_weights	13410	47	0.8372	0.2499	0.7468	0.4393	0.2579	34
Unet	vgg16	tversky_loss	default	13291	51	0.8363	0.0931	0.6417	0.2835	0.1981	21
Unet	resnet101	dice_loss	dynamic_weights	15050	59	0.8330	0.2612	0.7101	0.4223	0.2155	35
FPN	resnet50	dice_loss	dynamic_weights	15179	58	0.8309	0.2645	0.7083	0.3933	0.1956	26
Unet	mobilenetv2	dice_loss	dynamic_weights	15187	60	0.8289	0.2676	0.7347	0.4418	0.2240	25
Unet	resnet50	dice_loss	dynamic_weights	9946	33	0.8287	0.2666	0.7247	0.4153	0.2304	25
FPN	mobilenetv2	dice_loss_weighted	smaller_batch	18336	74	0.8286	0.5471	0.7089	0.3850	0.2268	27
FPN	mobilenetv2	dice_loss	dynamic_weights	19161	90	0.8214	0.2612	0.6997	0.3817	0.2148	22
DeepLabV3+_Custom	Xception	dice_loss_weighted	dynamic_weights	21341	85	0.8080	0.6289	0.6852	0.2566	0.1562	32
DeepLabV3+_Custom	Xception	dice_loss	default	18545	80	0.8016	0.3023	0.6865	0.2614	0.1638	27
DeepLabV3+_Custom	Xception	tversky_loss	dynamic_weights	16567	61	0.7850	0.1252	0.6677	0.2226	0.0000	29
PSPNet	resnet50	dice_loss	PSPNet_image_size	9951	40	0.7728	0.3285	0.7365	0.3818	0.1758	25
Unet	vgg16	categorical_crossentropy	default	8640	29	0.7635	0.3147	0.7583	0.3440	0.2190	22
Unet	vgg16	focal_loss	default	5957	16	0.5850	0.4584	0.4443	0.3837	0.1285	20



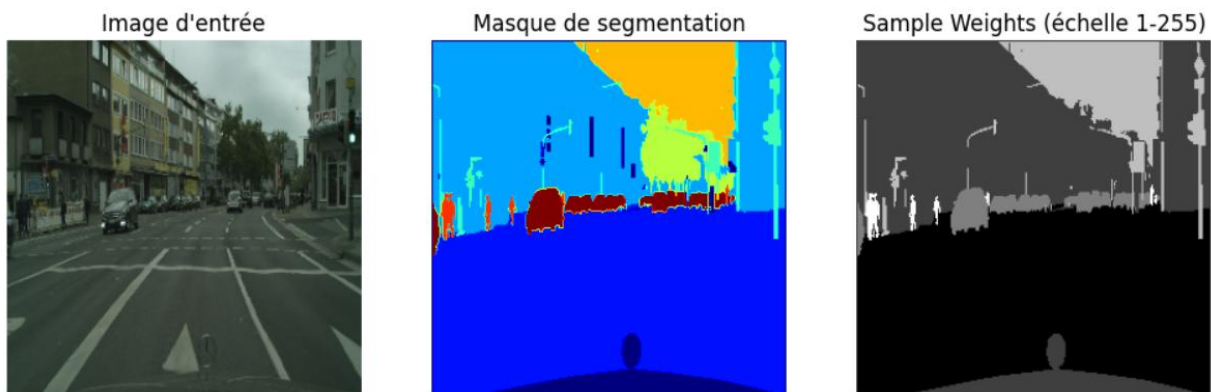
Classe void: IoU = 0.0737
 Classe flat: IoU = 0.8859
 Classe construction: IoU = 0.7285
 Classe object: IoU = 0.2240
 Classe nature: IoU = 0.6404
 Classe sky: IoU = 0.8081
 Classe human: IoU = 0.4418
 Classe vehicle: IoU = 0.7089



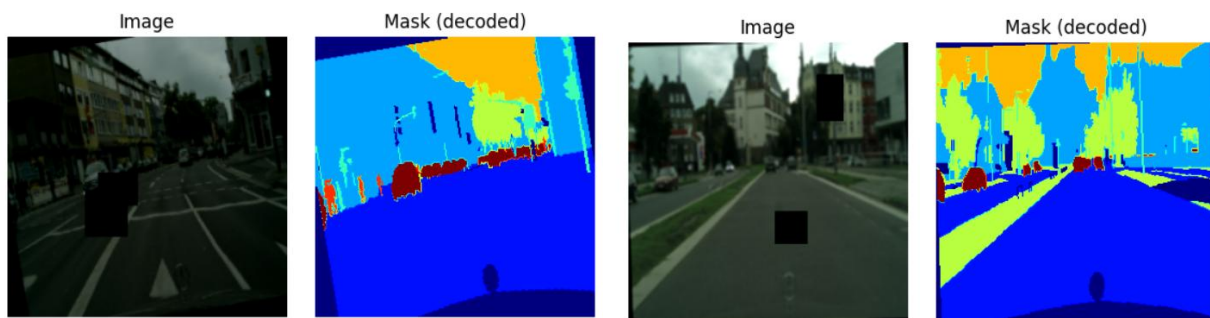
Annexe 2 : DataGenerator et options

Le DataGenerator personnalisé hérite de la classe `Sequence` de Keras. Il est conçu pour être flexible, rapide et adapté aux contraintes de segmentation avec classes déséquilibrées. Voici ses principales caractéristiques :

- Lecture dynamique des images/masques avec redimensionnement et normalisation
- Encodage one-hot des masques pour 8 classes
- Augmentations complexes avec Albumentations (flip, rotation, échelle, distorsions, météo...)
- Option `augmentation\_ratio` pour contrôler la part des données augmentées
- Gestion des poids de classes avec pondération logarithmique et ajustement progressif
- Support des poids d'échantillons (sample_weights) pour adapter la fonction de perte
- Méthodes `clone\_with\_modifications` et `from\_preset` pour tester facilement plusieurs configurations



Vérification des cartes de poids (jeu d'entraînement)



Vérification de la génération de données avec augmentation (jeu d'entraînement)

Annexe 3 : Fonctions de perte testées

Les fonctions de perte utilisées ont été choisies pour leur capacité à traiter le déséquilibre entre classes :

- **Cross Entropy** : mesure standard de classification pixel à pixel. Pour chaque pixel, on prédit une probabilité d'appartenance à chaque classe. La cross entropy compare cette distribution prédite à la vraie (comparaison de vecteurs encodés one-hot = 1 pour la bonne classe, 0 pour les autres). Son principal désavantage est que chaque pixel est traité indépendamment des autres, donc dans le cas de classes déséquilibrées les classes majoritaires domineront la perte, ce qui rendra difficile l'apprentissage sur les classes rares.

La formule générale pour un pixel :

$$CE(y, \hat{y}) = - \sum_{c=1}^C y_c \cdot \log(\hat{y}_c)$$

- C : nombre de classes
 - y_c : 1 si le pixel appartient à la classe c , 0 sinon
 - $\hat{y}_c$: probabilité prédite pour la classe c
- **Dice Loss** : accentue les zones de recouvrement entre prédiction et vérité terrain (1 - Dice coefficient). Elle mesure directement le recouvrement entre la segmentation prédite et la vérité terrain (GT), favorisant ainsi une meilleure performance sur des classes sous-représentées.

$$\text{Dice Coefficient} = \frac{2 \sum (P \cdot T)}{\sum P^2 + \sum T^2}$$

où :

- P est la prédiction du modèle (valeurs continues entre 0 et 1 après la sigmoïde ou softmax),
- T est le masque de vérité terrain (valeurs binaires 0 ou 1).

La **Dice Loss** est simplement :

$$\mathcal{L}_{\text{Dice}} = 1 - \text{Dice Coefficient}$$

- **Dice Loss pondérée** : applique un poids plus fort aux classes rares (en utilisant un vecteur de répartition global des poids).
- **Tversky Loss** : généralise Dice avec deux paramètres α et β pour ajuster l'importance de FP/FN.
 - $|A \cap B|$ représente toujours les **vrais positifs (TP)**.
 - $|A \setminus B|$ représente les **faux positifs (FP)**, c'est-à-dire les pixels prédits comme objets mais absents dans la vérité terrain.
 - $|B \setminus A|$ représente les **faux négatifs (FN)**, c'est-à-dire les pixels présents dans la vérité terrain mais manquants dans la prédiction.
 - α et β sont des poids qui ajustent l'importance des **FP** et **FN**.

La Tversky Loss est donc :

$$\text{Tversky Loss} = 1 - \frac{|A \cap B|}{|A \cap B| + \alpha |A \setminus B| + \beta |B \setminus A|}$$

Si $\alpha = \beta = 0.5$, alors la Tversky Loss devient identique à la Dice Loss.

Si $\alpha > \beta$, la perte pénalise davantage les **faux positifs** (utile pour éviter les sur-prédictions).

Si $\beta > \alpha$, la perte pénalise davantage les **faux négatifs** (utile pour éviter les sous-prédictions).

- **Focal Loss** : focalise l'apprentissage sur les prédictions difficiles via un facteur d'échelle (α, γ) .

Formule de la Focal Loss

La formule de la Focal Loss est la suivante :

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

où :

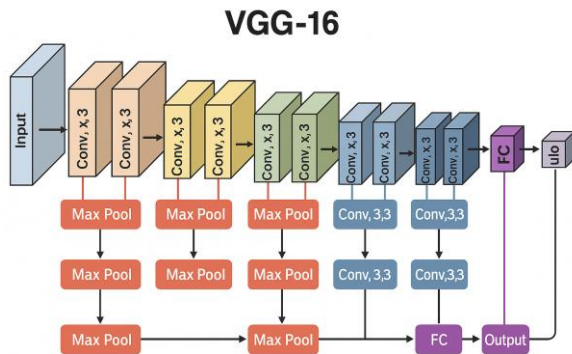
- p_t est la probabilité prédite pour la classe correcte (par exemple, $p_t = y \cdot \hat{p} + (1 - y) \cdot (1 - \hat{p})$ pour une classification binaire, où y est l'étiquette réelle et $\hat{p}$ la probabilité prédite).
- α_t est un facteur de pondération de la classe (permet de rééquilibrer les classes, notamment si les classes sont déséquilibrées).
- γ est un paramètre de modulation qui ajuste l'importance des exemples difficiles : une valeur élevée (par exemple, $\gamma = 2$) met davantage l'accent sur les exemples difficiles.

Avantages de la Focal Loss

1. **Réduction de l'impact des exemples faciles** : En utilisant le terme $(1 - p_t)^\gamma$, la Focal Loss donne moins de poids aux exemples facilement classifiables (où p_t est proche de 1 ou 0), et augmente le poids des exemples plus difficiles (où p_t est proche de 0.5).
2. **Gestion des déséquilibres de classes** : Le facteur α_t permet de donner plus de poids aux classes sous-représentées, ce qui est utile dans les cas où certaines classes sont rares, comme dans la détection d'objets ou la segmentation sémantique.
3. **Amélioration des performances sur des tâches difficiles** : En focalisant l'attention du modèle sur les erreurs, la Focal Loss améliore la capacité du modèle à se concentrer sur les objets difficiles à prédire, souvent négligés dans les approches traditionnelles.

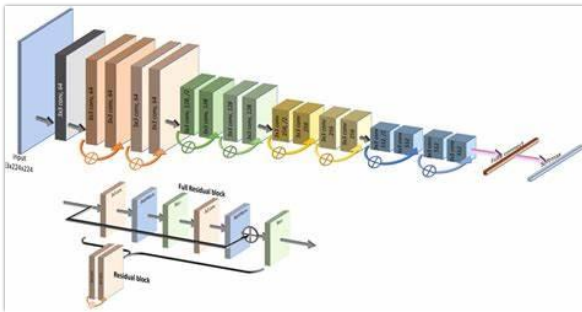
Annexe 4: Backbones testés et spécificités

- VGG16 : architecture simple, efficace, peu profonde. C'est une succession de blocs de convolution 3x3 suivis de pooling qui diminuent progressivement les dimensions. Convient pour des bases légères mais moins performante sur les détails fins. C'est un modèle assez gourmand en ressources.



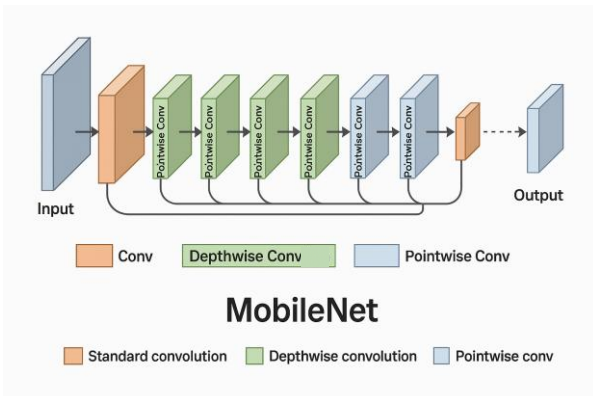
Modèle VGG16 complet

- ResNet50/101 : ajout de résidus pour l'apprentissage profond. Plus robuste mais plus lent, améliore les détails complexes grâce à la limitation de la perte de gradient.



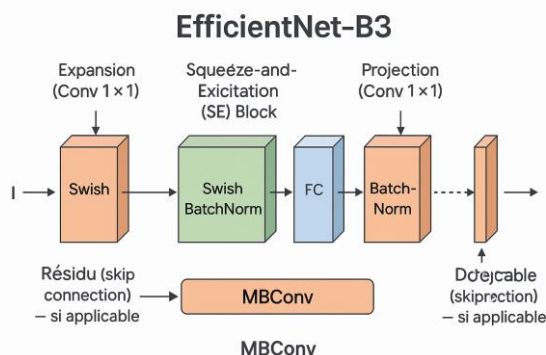
Principe des connexions résiduelles

- MobileNetV2 : très léger, rapide, adapté à un usage embarqué. Idéal pour réduire la taille du modèle (< 100 Mo).



Principe de convolution séparable en profondeur (filtre distinct sur chaque canal d'entrée) / convolution ponctuelle (combine les sortie DSC)

- EfficientNetB3 : équilibré entre précision et efficacité, mais plus lourd et plus lent à l'inférence.

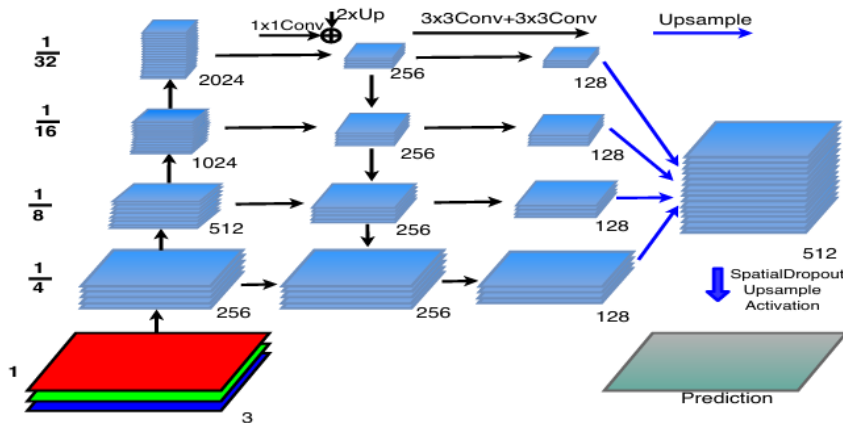


Optimisation multidimensionnelle (profondeur, largeur, résolution) avec des blocs MBConv :

Convolution par expansion (1x1) / convolution séparée en profondeur (kxk) / bloc d'attention (Squeeze-and-Excitation) / projection (1x1) et connexion résiduelle pour améliorer l'efficacité tout en maximisant les performances.

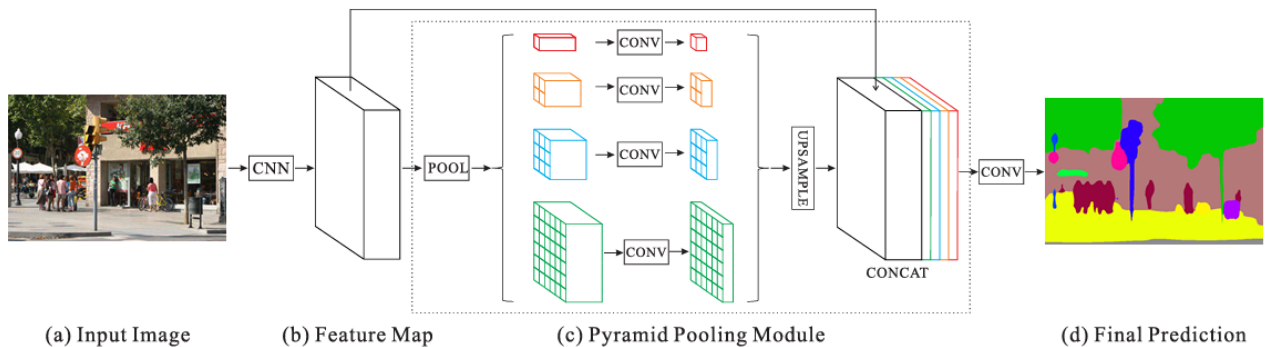
Annexe 5 : Modèles alternatifs et applications

- FPN (Feature Pyramid Network) : intègre des informations multi-échelles, très utile pour la détection d'objets de tailles variées. Il extrait à différents niveaux de l'encodeur des features à des résolutions différentes qui sont combinées à celles des couches de up-sampling (après convolution).



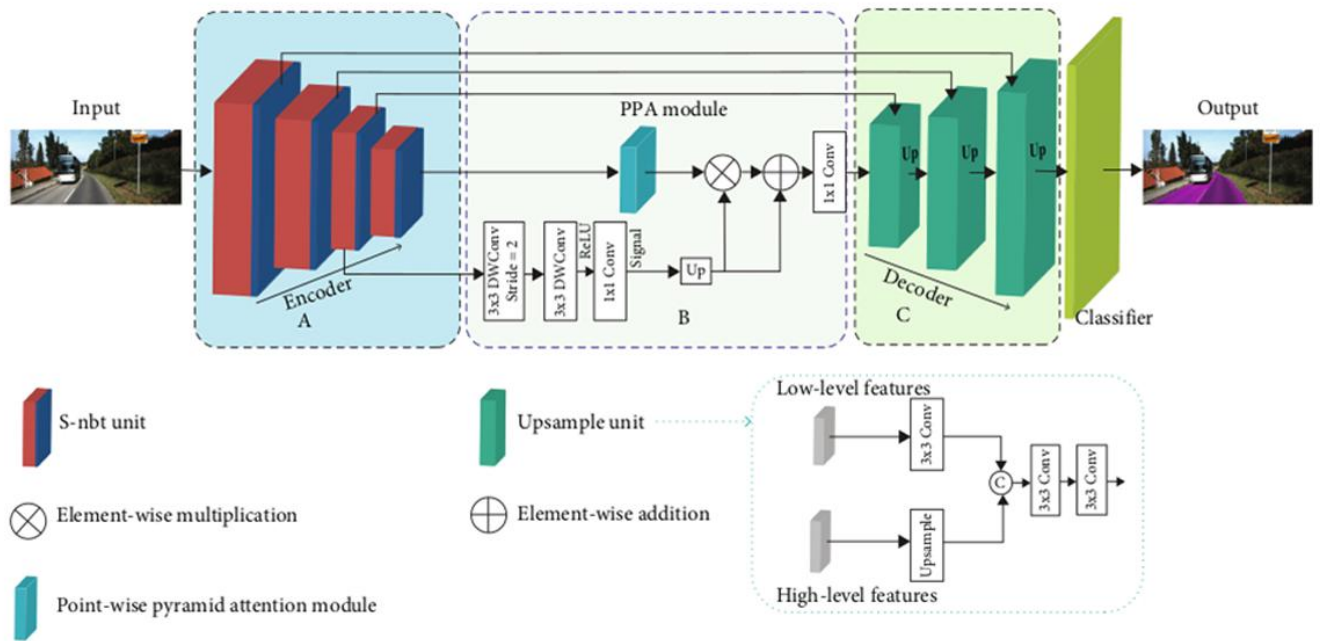
(source : arXiv:1806.03510v1 [cs.CV] 9 Jun 2018)

- PSPNet : segmentation pyramidale, capture le contexte global. Bon compromis précision / complexité, utile en environnement urbain.



(source : arXiv:1612.01105v2 [cs.CV] 27 Apr 2017)

- DeepLabV3+ : segmentation avec convolutions à trous (dilated), très performant sur les bords. Le module ASPP applique des convolutions dilatées (atrous convolutions) avec plusieurs taux de dilation. Cela permet de capturer des informations à différentes échelles sans réduire la résolution spatiale de la feature map. Combine également la détection des détails en extrayant les caractéristiques de haut niveau mais également celles extraites au début de l'encodeur.



(source : preprint DOI <https://doi.org/10.21203/rs.3.rs-2928293/v1> « Semantic Segmentation Algorithm for Autonomous Driving using UNET Architectures: A Comparative Study »)